FINAL ASSIGNMENT · TECHNICAL GUIDE

CodeMap AI

An agentic, RAG-powered codebase intelligence tool — how it actually works.

Turn any repository into an interactive dependency graph, then ask an AI agent how it works.

This guide explains every stage of the system end to end: how source code becomes a graph, how the graph is searched, and how the agent turns a question into cited answers — including the design decisions and the failure modes that shaped them.

The mental model

Three ideas compose the whole system:

- **Parse, don't guess.** A real parser (tree-sitter) extracts structure from code. We never regex a codebase hoping for the best.
- **Retrieve, don't stuff.** The LLM never sees the repository. It sees a handful of precisely retrieved chunks, chosen by hybrid search.
- **Traverse, don't paraphrase.** The graph answers questions a text search cannot — "what breaks if I change this?" is a graph query, not a similarity match.

> **Why this matters:** a generic RAG chatbot can summarise a README. It cannot tell you the *blast radius* of editing `crud.py`. That requires structure, and structure is what the parsing pipeline preserves.

The pipeline

```
Source code
   │
   ├─ 1. Parse        tree-sitter → entities (functions, classes, imports)
   │
   ├─ 2. Graph        entities → typed edges (contains / import / call)
   │                  + cycle detection, hub ranking, impact closure
   │
   ├─ 3. Chunk        entities → AST-aware chunks with a context header
   │
   ├─ 4. Index        chunks → ChromaDB (Jina embeddings, 768-d)
   │
```

```
└─ 5. Agent        question → ReAct loop → tools → cited answer
                   (search, read file, definitions, references, path, impact)
```

STAGE 1

Parsing with tree-sitter

tree-sitter builds a concrete syntax tree from source. We run S-expression *queries* against that tree to pull out the entities we care about. It is fast (C engine), tolerant of broken code, and works across languages with one API.

```
("function", "(function_definition name: (identifier) @name) @def")
("class",    "(class_definition name: (identifier) @name) @def")
("import",   "(import_from_statement) @def")   # relative imports live here
```

Each entity becomes a `CodeEntity`: name, type, start/end line, source snippet, file path.

> **A real bug this caught.** The original Python query matched `dotted_name`. But `from . import x` parses as a `relative_import` node — a different type — so **every relative import was silently dropped**. The graph looked plausible and was quietly wrong. Fixing the query and parsing the statement text recovered them.

Pattern-per-language, not one-size-fits-all

Language	Function pattern	Notes
Python	`function_definition`	plus `class_definition`, `import_from_statement`
JavaScript	`function_declaration`, arrow in `variable_declarator`	also TS/TSX via the TS grammar
Java	`method_declaration`, `constructor_declaration`	interfaces count as classes
Go	`function_declaration`, `method_declaration`	structs via `type_spec`
Rust	`function_item`	struct/enum/trait/impl
C / C++	`function_definition` → `function_declarator`	`#include` both quoted and angle-bracket

Each grammar's queries compile independently, so one mismatch degrades one pattern rather than losing a language.

STAGE 2

Building the graph

The graph is a `networkx.DiGraph`. The single most important decision is that **edges are typed**:

Edge kind	Meaning	Counted as a dependency?
`contains`	file → the entity defined inside it	**No**
`import`	file → file (resolved) or file → external module	Yes
`call`	function → function it invokes	Yes

> **Why typing matters.** Before this, a file "depended on" every function it contained. Degree counts, hub rankings, and the impact closure were all inflated — a file's own contents made it look like a dependency hub. Metrics now run over dependency edges only.

Import resolution

Every import is resolved against the project root, because paths on disk are absolute while module paths are not.

```
# Python
from .helper import thing      →  pkg/helper.py          (relative, walks up by dot count)
from . import helper           →  pkg/helper.py          (imported name is the submodule)
from pkg.mod import run        →  pkg/mod.py             (absolute, with suffix match)
import os                      →  (external)

# JavaScript / TypeScript
import x from './util'         →  src/util.js   (tries .ts .tsx .js .jsx, then /index.*)
import react                   →  (external — bare specifier means node_modules)
```

Resolution tries the full dotted path, then progressively shorter prefixes — so `from pkg.mod import name` works whether `name` is a module or a symbol.

Measured impact: on the `itsdangerous` repository, resolved imports went from 13 to 36, and the hub list changed from noise to the actual core (`exc.py` , `encoding.py` , `signer.py`).

Impact analysis (the blast radius)

A breadth-first traversal over *reverse* dependency edges:

```
seen = {target: 0}
queue = [target]
while queue:
    current = queue.popleft()
    for source in reverse_edges(current):     # who imports / calls me
        if source not in seen:
```

```
            seen[source] = seen[current] + 1  # distance in hops
            queue.append(source)
```

The result is every file affected by a change, ranked by distance. This is the one question the graph answers that retrieval never can.

STAGE 3

AST-aware chunking

Naive chunking splits on characters or line counts, cutting functions in half. We chunk on *entity boundaries* instead, and prefix each chunk with a header that gives the embedding model context about where the code lives:

```
# File: routers/todos.py | Type: function | Name: read_tasks | Lines: 19-24
def read_tasks(db: Session, owner_id: int, *, done: bool | None = None) -> list[Task]:
    stmt = select(Task).where(Task.owner_id == owner_id)
    ...
```

The header buys two things: better embeddings (the model sees the file and symbol, not an orphan block), and **precise citations** — line numbers are carried into metadata.

STAGE 4

Hybrid retrieval

Embeddings alone are weak at exact identifiers. Ask for `get_db` and a vector search may return conceptually similar code with a different name. So we run two searches and fuse them.

Semantic (vector)

Jina `jina-embeddings-v2-base-code` in ChromaDB. Strong on "where is authentication handled", weak on exact names.

Lexical (BM25)

Okapi BM25 over the chunk corpus. Strong on exact identifiers, blind to meaning.

Results are merged with **reciprocal rank fusion**, which combines rankings without needing the two score scales to be comparable:

```
score[file] += 1 / (RRF_K + rank)      # RRF_K = 60, summed across both rankings
```

Finally we keep **one chunk per file**, so five results mean five different files rather than five slices of one.

STAGE 5

The agent

A ReAct (Reason → Act → Observe) loop. The model is given tools and decides which to call; we execute them and feed results back until it answers.

The seven tools

Tool	Answers

`search_codebase(query)`	hybrid search over the index
`get_file_content(filepath, start?, end?)`	read a file (200 lines by default)
`find_definition(symbol)`	where is this defined?
`find_references(symbol)`	what imports or calls this?
`get_dependency_path(a, b)`	how are these two connected?
`impact_analysis(node)`	what breaks if this changes?
`generate_diagram(components)`	Mermaid architecture diagram

Everything streams to the browser over Server-Sent Events as typed events: `action`, `observation`, `highlight`, `chunk`, `done`. The UI turns these into the live status line — and, for `impact_analysis`, into the graph highlight.

Failure modes and how they're handled

These are the real problems encountered while building, and the guards that came out of them. They matter more than the happy path.

1 Local models emit tool calls as text

Some models stream `{"name": "search_codebase", ...}` as content instead of a native call. Untreated, that JSON appears verbatim in the answer.

Guard: match the tool-call marker mid-stream and suppress it, holding back 12 trailing characters so a marker split across chunks cannot leak.

2 Providers reject an unparseable tool call

Groq returns HTTP 400 `tool_use_failed` — sometimes while streaming, not at request time.

Guard: two recovery steps. First drop the tools and flatten the transcript; if that still fails, send a minimal transcript with no tool exchange at all.

3 The request exceeds the token ceiling

Tool output is replayed on every iteration, so a few file reads blow a per-minute limit (HTTP 413).

Guard: cap chunks (900 chars) and file reads (200 lines); compact the transcript to the most recent tool rounds; retry once with a trimmed request. Worst case measured at ~3,000 tokens against an 8,000 ceiling.

4 Free tiers run out mid-session

Groq's daily cap is 200,000 tokens — a dozen questions exhausts it.

Guard: an optional fallback provider, switched to automatically on a rate-limit error. Provider errors become actionable messages ("daily token limit used up") rather than raw JSON.

5 Answers rendered badly in a narrow panel

Tables broke because code like `mode: str | None` contains a pipe, which splits a markdown table cell.

Guard: the system prompt forbids pipes inside table cells and prefers bullets; tables scroll horizontally; the Explain prompt asks for a brief summary, not a report.

Frontend architecture

Route / component	Responsibility
`/`	Landing: analyse a path or GitHub URL, load sample, recent projects
`/project/[id]`	Workspace: graph, code panel, connections, chat
`/project/[id]/chat`	Full-width reading view
`ForceGraph`	D3 force layout; filters, trace path, blast-radius spotlight
`CodeViewer`	Syntax-highlighted source, cited-line highlight, Explain

`ChatPanel` / `useAgentChat`	SSE stream parsing, status line, clickable citations

State lives in the route components; the graph and code panel are pure presentational children driven by props.

Quick reference: where things live

```
backend/
  main.py                     app, .env loading, /api/health
  parser/tree_sitter_parser.py   multi-language entity extraction
  parser/graph_builder.py        typed edges, resolution, impact, insights
  parser/chunker.py              AST-aware chunks + headers
  rag/embeddings.py              ChromaDB client + Jina embeddings
  rag/retriever.py               hybrid search (vector + BM25, RRF)
  agent/react_agent.py           ReAct loop, retries, fallbacks
  agent/tools.py                 the seven tools
  routers/                       upload, clone, graph, chat, overview, export
frontend/src/
  app/page.tsx                   landing
  app/project/[id]/page.tsx      workspace route
  components/Workspace.tsx       workspace shell + panel orchestration
  components/ForceGraph.tsx      D3 graph
  hooks/useAgentChat.ts          SSE client
```

Running it

```
# Backend
cd backend
cp .env.example .env      # set AI_MODEL to a model your endpoint serves
uvicorn main:app --port 8000
curl localhost:8000/api/health

# Frontend
cd frontend && npm run dev
# open http://localhost:3000
```

> `.env` is the single source of truth for LLM configuration and overrides shell variables, so the base URL, key, and model always move together. A mismatch here is the most common cause of empty answers.